FDD — Sistema Web CAD/CAM de Marcenaria Paramétrica e Gestão

Documento de escopo (Feature Driven Development) com todas as fases do projeto.

- Repositório: github.com/Christian0FS/Sistema-de-Marcenaria-com-Modelador-3D
- Pasta local: `C:\Users\usuario\OneDrive\Desktop\Projetos\Mateus` (`frontend/` e `backend/`)
- Inspiração: Promob / PlanIT

Legenda de status

Símbolo	Significado
✅	Concluída e testada
🟡	Código entregue, falta aplicar, integrar ou validar
⬜	Pendente
💡	Proposta, ainda não confirmada

---

1. Visão geral

Sistema web para planejar móveis planejados de forma paramétrica: o marceneiro define as dimensões do módulo, divide em vãos, coloca prateleiras, gaveteiros e portas, e o sistema evolui até gerar plano de corte, arquivos de máquina, orçamento automático e contrato.

Dois painéis distintos:

1. **Painel de Insumos**: cadastro de chapas MDF, fitas de borda e ferragens (com preço e estoque).
2. **Painel de Projetos**: montagem 3D dos móveis consumindo os insumos cadastrados.

Fluxo do software de referência analisado: Estrutura (L/A/P, divisões, internos, portas) → Aparência → Visualizador 3D → Orçamento (plano de corte com % de aproveitamento, export Excel/DXF/XML) → Contrato de compra → Vista explodida numerada.

2. Stack e arquitetura

Camada	Tecnologia
Frontend	React 17, Tailwind CSS, Zustand, React Three Fiber / Three.js, @dnd-kit, React Router
Backend	Python 3.12, FastAPI, Pydantic, JWT (python-jose), bcrypt direto
Banco	Turso (libSQL/SQLite na nuvem) via `libsql-client`, SQL puro, sem ORM
Outras libs	reportlab, ezdxf, qrcode

Arquitetura: o React fala com o FastAPI por HTTP; só o backend acessa o banco. Os módulos 3D ficam serializados em JSON dentro do projeto.

3. Modelo de dados (7 tabelas)

Tabela	Conteúdo
`usuarios`	Login (id UUID em texto)
`clientes`	Nome, telefone, e-mail, endereço
`projetos`	Cliente, nome, status (lead/orcamento/producao/montagem), `modulos_json`
`chapas_mdf`	Chapas MDF com preço e estoque
`fitas_borda`	Fitas de borda
`ferragens`	Tipos válidos hoje: dobradica, corredica, puxador, minifix, cavilha
`biblioteca_modulos`	Modelos reutilizáveis por usuário (`estrutura_json`, categoria, thumbnail)

Todos os IDs são UUID em texto (`str(uuid.uuid4())`), não inteiros autoincrementais.

Formato de um módulo (dentro de `modulos_json`):

```
{ modulo_id, nome, dimensoes{largura,altura,profundidade},
  estrutura{ divisoes[], internos[], portas[] }, aparencia{cor,
orientacao_fibra} }
```

4. Regras de negócio

1. **Vãos recursivos**: um vão pode ser dividido na vertical ou horizontal; cada divisão gera dois vãos filhos. O painel divisório ocupa 15 mm, centrado na posição escolhida.

2. **Prateleiras empilháveis**: um vão aceita várias prateleiras. N prateleiras geram N+1 compartimentos de altura igual (redistribuição automática). Ex.: guarda-sapato.
3. **Gaveteiro exclusivo**: um vão com gaveteiro não recebe prateleira nem divisão. Folga de 3 mm entre gavetas.
4. **Divisão exclusiva**: um vão que já tem prateleira/gaveteiro não pode ser dividido.
5. **Portas**: só em vão sem divisão. Tipos: giro (lado da dobradiça), corrediça (par de folhas com 15 mm de sobreposição) e báscula. Folga padrão de 3 mm em todo o contorno, espessura de 18 mm.
6. **Ferragens por peça**: giro 2 dobradiças (3 se a altura passa de 1600 mm) + 1 puxador; corrediça 1 kit de trilho + 2 puxadores; báscula 2 dobradiças de báscula + 1 puxador; gaveta 1 par de corrediças (tamanho = profundidade do módulo) + 1 puxador.
7. **Chapas**: cada peça referencia uma chapa por `chapaId`. A cor mora no cadastro da chapa (color picker), a cor do veio é calculada (~20% mais escura). Isso alimenta nesting e preço.
8. **Biblioteca**: cada marceneiro vê só a própria biblioteca. Ao carregar um modelo em outras dimensões, a estrutura é reescalada proporcionalmente.

5. Roadmap por fases

Fase 1 — Fundação ✅

Etapa	Descrição	Status
1.1	Setup do projeto, banco e migração	✅
1.2	Autenticação JWT (register/login)	✅
1.3	CRUD de clientes e projetos	✅
1.4	Painel de Insumos (chapas, fitas, ferragens)	✅
1.5	Kanban com drag-and-drop (Leads, Orçamentos, Produção, Montagem)	✅

Fase 2 — Módulo 1: CAD paramétrico

Etapa	Descrição	Status
2.1	Canvas 3D e caixa paramétrica (L/A/P)	✅
2.2	Persistência de módulos no backend	✅
2.3	Divisórias recursivas	✅
2.4	Prateleiras e gaveteiros	✅ (empilhamento de prateleiras: 🟡)
2.5	Portas com folga (giro, corrediça, báscula)	✅
2.6	Textura MDF com fibra via shader	🟡 shader entregue; os componentes 3D ainda usam `meshStandardMaterial`
2.7	Biblioteca de módulos reutilizáveis	✅ backend; 🟡 frontend (reescala depende de salvar as dimensões)
2.8	Animação de abertura de portas e gaveteiros	💡
2.9	Peças estruturais reais (laterais, topo, base, fundo)	💡 bloqueante para as Fases 3 e 4

Detalhes das etapas abertas:

- **2.8**: gaveta desliza no eixo Z; porta de giro rotaciona na dobradiça; báscula gira na base; corrediça desliza no eixo X. Serve para conferir interferências e para apresentação.
- **2.9**: hoje a caixa externa é só wireframe de referência. Transformar em chapas reais de MDF (usando o shader e o `chapaId`), com fundo configurável (3 a 6 mm, opcional). Sem isso não é possível adicionar painel de fundo (ex.: painel de TV) e as Fases 3 e 4 não enxergam essas peças.

Fase 3 — Módulo 2: Regras construtivas

Etapa	Descrição	Status
3.1	Vínculo de ferragens às peças	🟡 lógica e rota criadas; falta decisão de tipos e integração no frontend
3.2	Furação sistema 32 mm	⬜
3.3	Fita de borda e metragem linear	⬜

- **3.1 abertos**: (a) decidir se amplia os tipos válidos de ferragem (`dobradica_bascula`, `trilho_corredica`, `corredica_gaveta`) ou simplifica a lógica para os 5 tipos atuais; (b) popular o campo `ferragens` de cada peça no frontend antes de salvar; (c) seção "Ferragens vinculadas" no editor, com troca por dropdown.
- **3.2**: grid de 32 mm a 37 mm da borda frontal; dobradiças em posições próprias. Pergunta em aberto: prateleiras fixas ou ajustáveis (define se gera a linha inteira de furos).
- **3.3**: mapear faces com fita para calcular metragem linear.

Fase 4 — Módulos 3 e 4: CAM e precificação

Etapa	Descrição	Status
4.1	Bin packing 2D (plano de corte)	⬜
4.2	Exportação DXF	⬜
4.3	XML/CSV por máquina (INMES, Giben, SCM Cyflex, Corte Cloud)	⬜
4.4	Etiquetas com QR Code	⬜
4.5	Precificação automática (MDF + fita + ferragens + perda + margem de lucro)	⬜

Fase 5 — Módulo 5: Comercial

Etapa	Descrição	Status
5.1	Contrato/orçamento em PDF	⬜
5.2	Assinatura digital	⬜
5.3	Manual de montagem (vista explodida) acessado por QR	⬜
5.4	Visualização em ambiente/cenário (etapa pesada)	💡

- **5.4**: chão e parede configuráveis, vários módulos lado a lado, câmeras pré-definidas, props decorativos fixos (planta, cortina, tapete) para escala, iluminação mais realista (HDRI) e exportação de imagem para o material comercial.

Fase 6 — App mobile com Ionic/Capacitor (condicional)

Só entra se o cliente pedir. Tecnicamente viável: o React Three Fiber roda em WebView, e o código React atual é reaproveitado.

- Custos de loja: Google Play US$ 25 (único); Apple US$ 99/ano. Publicar no iOS exige um Mac (Xcode).
- O backend precisa estar hospedado com HTTPS (não pode ser `localhost`); a URL da API vira variável de ambiente do build.
- Sem internet o app não funciona (modo offline fica fora do escopo).
- O editor 3D precisa de layout revisado para toque/tablet.

6. Itens transversais

Item	Status
Tela de Login no frontend (hoje o token é colado manualmente no `localStorage`)	⬜
Tela de detalhe do projeto (listar os módulos do projeto, "+ Novo módulo")	⬜
Clique no card do Kanban abrir o editor (`navigate` + `activationConstraint`)	🟡
Rotas do `App.jsx` (`/`, `/editor/:projetoId`, `/editor/:projetoId/:moduloId`, `/biblioteca`)	🟡
Campo de cor no cadastro de chapas + `chapaId` nas peças	⬜ (junto da 2.9)
Deploy: Hostinger compartilhada só serve estático; backend em VPS Hostinger (Nginx + Gunicorn/Uvicorn + Let's Encrypt) ou Render/Railway, com frontend estático na Hostinger	⬜

7. Riscos e dependências

1. A etapa 2.9 precisa vir antes de 3.2, 4.1 e 4.5, porque todas assumem que as peças estruturais existem.
2. O `chapaId` por peça é dado central: afeta visual (2.6), nesting (4.1) e preço (4.5).
3. Reescala de modelos da biblioteca exige salvar as dimensões originais junto com a estrutura.
4. Os tipos de ferragem precisam ser fechados antes de a precificação consumir esse vínculo.